

Canes CFB Analytics

Predicting college football points, spreads and totals — and testing them against the market

Daniel Regalado

September 23, 2026

Table of contents

Summary	5
What the project does	5
Key results (as of 2026-09-23)	5
How to read this book	6
I Foundations	7
1 The project and the repository	8
1.1 Goal and scope	8
1.2 Markets and periods	8
1.2.1 Conventions	8
1.3 Repository layout	9
1.4 Tooling and working rules	9
2 Data	11
2.1 Sources	11
2.2 Ingestion and caching	11
2.3 Validation	12
2.4 Coverage of lines and advanced stats	12
3 What the data looks like	14
3.1 Is there enough data?	14
3.2 The target	14
3.3 Scoring is not stable over time	16
3.4 Home field and quarters	16
II Features and models	17
4 Feature engineering	18
4.1 The as-of rule	18
4.2 Opponent-adjusted ratings	18
4.2.1 Tuning α and τ	19
4.3 Elo with a nonlinear margin of victory	19
4.4 Feature families	19
4.5 Which families earn their place?	20
5 Nonlinearities and interactions	22
5.1 First, a direct test: do interactions help at all?	22
5.2 Model lens: shapes of the main effects	22

5.3	Model lens: interaction strength	23
5.4	Market lens: where does the closing line miss?	23
5.5	The shootout-under effect	25
6	Models, tuning and the one-shot test	26
6.1	Splitting by time	26
6.2	The model zoo and tuning	27
6.3	Ensembles, validation and the final pick	27
6.4	The market layer, and why it was dropped	28
7	Halves and quarters	29
7.1	Accuracy by period	29
7.2	Game script: why Q4 doesn't follow team strength	29
7.3	Medians, not means	30
III	Betting	31
8	From predictions to bets	32
8.1	Market-implied points	32
8.2	Edges and results	32
8.3	Break-even and ROI at -110	32
8.4	Closing line value (CLV)	33
8.5	The pre-registered paper rules	33
8.6	Logged picks so far	33
8.7	The weekly pipeline	34
9	Decision log and corrections	35
9.1	Decisions	35
9.2	Corrections: things that were wrong and got fixed	36
9.3	What the evidence says, plainly	36
10	What's next	38
10.1	Open work	38
10.2	Period lines (#4): options and costs	38
10.3	Should this become a website?	39
	Appendices	40
A	Reproducing everything	40
A.1	Environment	40
A.2	Order of operations	40
A.3	Rendering this book	41
B	Glossary	42
	References	44

List of Figures

2.1	FBS teams per season, as identified from the scoreboard (2026 includes scheduled games). .	12
3.1	Distribution of team points, game totals and home margins (FBS vs FBS, 2015–2025). . . .	15
3.2	Share of games decided by exactly N points. Field goals (3) and touchdowns (7) dominate. .	15
3.3	Average game total by season (FBS vs FBS). The 2023 clock rule cut plays per game. . . .	16
4.1	Error as families are added (LightGBM on residual), against the league-average baseline and the closing line.	21
5.1	Mean SHAP value by feature decile: points the model adds to the ratings' prediction. . . .	24
5.2	Under rate vs the closing total: expected shootouts vs all games, by season. Dashed line: 52.4% break-even at −110.	25
6.1	Train, validation and test by season, and the walk-forward cross-validation folds inside the training window.	26
7.1	Share of a team's regulation points scored in Q4, by the team's expected margin (2019–2023). .	30

List of Tables

1.1	The market registry: 18 model notebooks.	8
2.1	Data sources.	11
2.2	Share of FBS-vs-FBS games with each data type, by season (%).	12
2.2	Share of FBS-vs-FBS games with each data type, by season (%).	13
3.1	Completed games by season, 2015–2025.	14
4.1	MAE of exp_points alone (team points, 2017–2023) for each shrinkage α and recency half-life τ (days).	19
4.2	Feature families and their sizes.	19
4.2	Feature families and their sizes.	20
4.3	Cross-validated MAE (points per team, 2019–2023) as feature families are added.	20
5.1	Walk-forward CV MAE: additive vs interaction-capable LightGBM.	22
5.2	Strongest pairwise SHAP interactions (mean absolute contribution, points).	23
5.3	Segments where the closing line missed in 2016–2023 ($ z \geq 2$), with the 2024–25 holdout.	24
6.1	Data splits.	26
6.2	Optuna results (mean per-season CV MAE, 2019–2023). Read live from models/team_points_params.json.	27
6.3	Model selection and the one-shot test (copied from the experiment log).	28
7.1	Walk-forward CV MAE (points per team, 2019–2023) by period.	29
8.1	Paper-trading rules.	33
8.2	Paper bets logged for 2026 (before kickoff).	34
9.1	Main decisions.	35
10.1	Open work.	38
10.2	Options for period lines.	39
B.1	Terms used in this book.	42

Summary

This book documents everything built in **canes-cfb-analytics** so far: the data, the features, the models, how they were evaluated, what worked, what didn't, and the mistakes that were caught and corrected along the way. It's written to be read top to bottom by someone who wasn't there, including future me.

What the project does

It predicts FBS college football games (winner, spread, total, team totals, plus halves and quarters) and asks one question of every model: **does it know something the betting market doesn't?** Accuracy alone isn't enough. A model can be accurate and still lose money if the market is more accurate.

The core model predicts **points per team**. Spread (the difference), total (the sum) and winner (who scores more) all derive from the two predictions for a game.

Key results (as of 2026-09-23)

Question	Answer
Enough data?	Yes. 10,630 games (2015–2026), 16,642 FBS-vs-FBS team-game rows for 2015–2025, points by quarter for every game.
Best single feature	Opponent-adjusted ratings (ridge regression refit every week): error falls from 11.1 to 9.44 points per team.
Best model	Tuned random forest on the ratings' residual. Test error (2025): 8.94 points per team.
Versus the market	The market is more accurate: 8.70 (opening line) and 8.59 (closing line).
Interactions?	Small. The signal lives in one-variable curves and thresholds (regression to the mean, talent-gap tails, a returning-production threshold), not in crosses.

Question	Answer
Betting edge	Not proven. Two pre-registered rules are paper-traded in 2026: A (bet the model's side when its total differs from the opener by ≥ 4) and B (bet the under when the ratings expect a shootout, which went 55–60% under in every season 2021–2025).
Real money?	No. A full season of paper results first.

How to read this book

- **Foundations** covers the repository, the data and what the data looks like.
- **Features and models** covers feature engineering, the search for nonlinearities, the model zoo, tuning, the one-shot test, and halves and quarters.
- **Betting** covers how predictions become bets, how they're graded, the weekly automation, the decision log (including corrections), and what's next.
- The appendices explain how to reproduce everything and define the jargon.

Every number in a table or figure comes from code run against the data, not typed in by hand, except where a chapter says a table is copied from an experiment log (the slow experiments: tuning and cross-validated ablations).

Part I

Foundations

1 The project and the repository

1.1 Goal and scope

The request that started it: a repository with **one folder per thing we predict**, each holding Jupyter notebooks, covering:

- the **winner** of each game,
- the **spread** (point difference),
- the **total** points of the game,
- the **total points of each team**,
- and spread and total **by halves and quarters**.

It lives at github.com/DanielRegaladoUMiami/canes-cfb-analytics (public, Apache 2.0). It's a separate project from `cfb-canes-analytics`, an earlier repository that prices full-game totals against Kalshi and Polymarket. This one benchmarks against sportsbook lines only.

1.2 Markets and periods

Every (market, period) pair is registered in one place, `src/canes_cfb/markets.py`, and the notebook tree is generated from it by `scripts/make_notebooks.py`.

Table 1.1: The market registry: 18 model notebooks.

Market	Folder	Full game	1H	2H	Q1-Q4
Winner (moneyline)	<code>notebooks/01_winner</code>	✓			
Spread	<code>notebooks/02_spread</code>	✓	✓	✓	✓
Game total	<code>notebooks/03_total</code>	✓	✓	✓	✓
Team totals	<code>notebooks/04_team_total</code>	✓	✓	✓	

1.2.1 Conventions

Everything is written from the **home team's perspective**:

$$\text{margin} = \text{home points} - \text{away points}, \quad \text{home covers} \iff \text{margin} + \text{spread} > 0,$$

where the spread is the home team's line (negative means home is favored). A game ends in a push when $\text{margin} + \text{spread} = 0$.

Periods follow standard US sportsbook rules: quarters are **regulation only**, $1H = Q1 + Q2$, and $2H = Q3 + Q4 + OT$, so that $1H + 2H$ equals the full game.

1.3 Repository layout

```
canes-cfb-analytics/
├─ notebooks/
│   ├─ 00_data/          ingest (ESPN, CFBD) → EDA → features → nonlinearity insights
│   ├─ 01_winner/ 02_spread/ 03_total/ 04_team_total/
│   └─ 99_evaluation/    backtest against the lines
├─ src/canes_cfb/        shared, tested code (every notebook imports it)
│   ├─ espn.py cfbd.py    data clients with on-disk caches
│   ├─ features.py periods.py  as-of features
│   ├─ modeling.py validation.py market.py betting.py
│   └─ markets.py paths.py
├─ scripts/              make_notebooks · tune_team_points · predict_week · grade_paper
├─ models/                tuned parameters + final recipe (JSON)
├─ paper_trading/         pre-registered picks and their graded results
├─ docs/                  pipeline, market rules, research, experiment logs
├─ report/                this book (Quarto)
├─ tests/                 34 tests, including leakage tests
└─ .github/workflows/     weekly automation
```

1.4 Tooling and working rules

Tool / rule	Why
uv for Python and dependencies (<code>pyproject.toml</code> , <code>uv.lock</code> committed)	Reproducible environments on any machine
ruff (lint + format) and nbstripout as pre-commit hooks	Consistent code; notebooks never carry outputs or data into git
Logic reused by two or more notebooks moves into <code>src/</code> with a test	Every model is evaluated the same way

Tool / rule	Why
As-of features only , splits by time , the closing line as benchmark	Honest evaluation (Chapter 4, Chapter 6)
Issues + milestones + ROADMAP.md + experiment logs	Lightweight tracking; decisions live in the repo, not in chat
Conventional commits, sole author	Clean history

Two operational details are worth knowing:

- **iCloud and virtual environments.** The repository sits under an iCloud-synced Desktop. iCloud marks files as hidden, and Python then ignores the environment's `.pth` files, so `import canes_cfb` fails with a phantom `ModuleNotFoundError`. The fix: `.venv` is a symlink to `.venv.nosync/`, a folder iCloud never syncs.
- **Secrets.** The CollegeFootballData API key lives in `~/.zshrc` locally and in a GitHub Actions secret for automation. The client reads it from the environment and never writes it to disk. So far, each commit was checked by hand to confirm the key isn't in any file (not yet an automated hook).

2 Data

2.1 Sources

Table 2.1: Data sources.

Need	Source	Access
Games, venue, neutral site, conference game, final score	ESPN scoreboard (groups=80, FBS)	Free, no key
Points by quarter (Q1–Q4, OT)	ESPN scoreboard linescores	Free, no key
Betting lines: spread, total, moneyline, open and close	CollegeFootballData /lines (<i>CollegeFootballData.com API</i> 2026)	Free key, 3,000 calls/month
Efficiency per team-game: plays, EPA/PPA, success rate, explosiveness, pass/rush splits	CFBD /stats/game/advanced	Same key
Talent composite, returning production	CFBD /talent, /player/returning	Same key

CFBD game ids are the same as ESPN’s, so every table joins on `game_id`. Tables keyed by school name (advanced stats, talent, returning production) map to ids through **FBS teams only**, because some names repeat across divisions (there’s more than one “Charlotte” and “Troy”).

2.2 Ingestion and caching

Both clients (`src/canes_cfb/esp.py`, `cfbd.py`) cache every raw response as JSON under `data/raw/`:

- **ESPN**: a week whose games are all final is never downloaded again. Weeks with unfinished games are refreshed.
- **CFBD**: one call per season per endpoint. Past seasons are never requested twice, and the current season is refreshed. A full rebuild of 2015–2026 costs ~60 calls.

The data itself is **never committed** (`data/` is git-ignored, ~220 MB). It can be rebuilt from the notebooks in `00_data/`, and republishing ESPN/CFBD data in a public repository isn’t appropriate.

2.3 Validation

Every check passes:

1. **One row per game.** `game_id` is unique across 10,630 games (9,764 completed).
2. **Quarters add up.** Q1 + Q2 + Q3 + Q4 + OT equals the final score in every completed, full-length game (0 mismatches).
3. **FBS membership is right.** A team counts as FBS in a season if it plays 6+ games in that season's FBS scoreboard; FCS opponents appear only in their one or two games against FBS teams. The counts match reality (128 teams in 2015, 134 in 2024).
4. **Weather-shortened games are flagged.** 3 games ended before Q4 (for example, Kentucky vs Southern Miss in 2024, called for lightning). They have no quarter or half targets and are excluded from training.

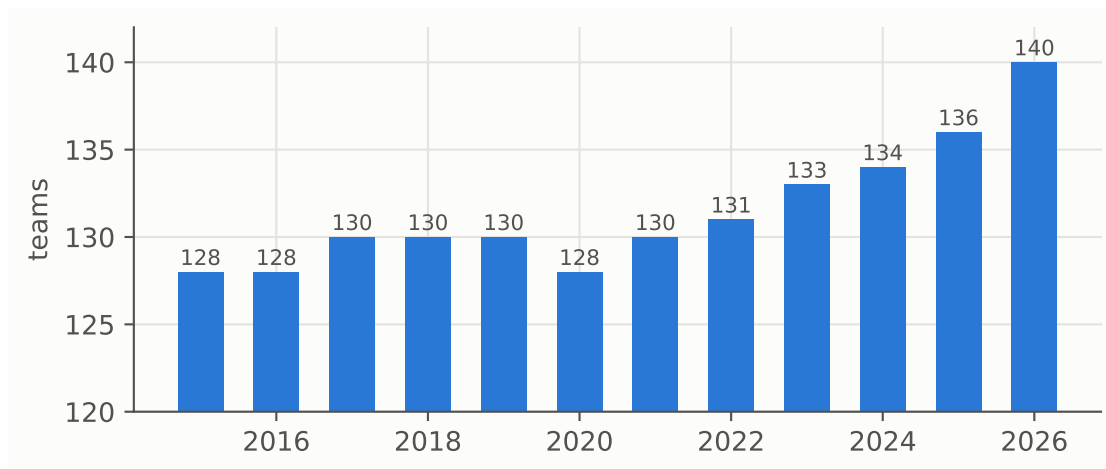


Figure 2.1: FBS teams per season, as identified from the scoreboard (2026 includes scheduled games).

2.4 Coverage of lines and advanced stats

Table 2.2: Share of FBS-vs-FBS games with each data type, by season (%).

season	games	closing spread	opening spread	advanced stats
2015	765	100.0	0.0	99.1
2016	760	100.0	0.0	98.3
2017	776	100.0	0.0	99.5
2018	772	100.0	0.0	99.7
2019	774	100.0	0.0	99.9
2020	530	100.0	0.0	100.0

Table 2.2: Share of FBS-vs-FBS games with each data type, by season (%).

	games	closing spread	opening spread	advanced stats
season				
2021	770	100.0	100.0	100.0
2022	776	100.0	99.1	100.0
2023	792	100.0	100.0	100.0
2024	798	100.0	100.0	99.5
2025	808	100.0	100.0	100.0

Two facts shape everything downstream:

- **Closing lines cover every game**, so the closing line is always available as the benchmark.
- **Opening lines only exist from 2021**. Anything trained *against the opener* has at most three seasons (2021–2023) before validation. That’s why the market layer in Chapter 6 is deliberately tiny.

Consensus lines are the **median across books** (ESPN Bet, DraftKings, Bovada in recent seasons).

3 What the data looks like

3.1 Is there enough data?

Table 3.1: Completed games by season, 2015–2025.

season	games	FBS vs FBS	vs FCS	team rows
2015	870	765	105	1530
2016	873	760	113	1520
2017	874	776	98	1552
2018	884	772	112	1544
2019	888	774	114	1548
2020	568	530	38	1060
2021	887	770	117	1540
2022	896	776	120	1552
2023	911	792	118	1584
2024	919	798	121	1596
2025	934	808	126	1616
total	9504	8321	1182	16642

The first model predicts **each team's points**, so every game gives two rows: 8,321 FBS-vs-FBS games become 16,642 training rows. That's plenty for linear models and enough for regularized gradient boosting, but it isn't big data. **Feature quality matters more than model complexity**, and that turned out to be the central lesson of the project.

Games against FCS teams (1,182, average margin ~31 points) are excluded from training. They're lopsided and not what we bet on. 2020 is small because of COVID cancellations.

3.2 The target

Team points average 27.7 with a standard deviation of 13.9. A model that always guesses the average misses by about 11 points. That's the floor every model has to clear.

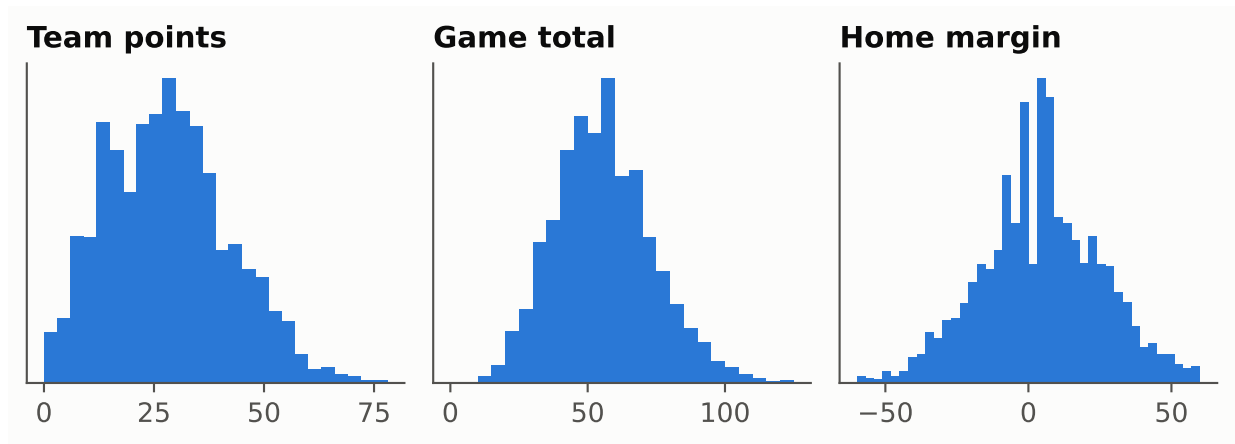


Figure 3.1: Distribution of team points, game totals and home margins (FBS vs FBS, 2015–2025).

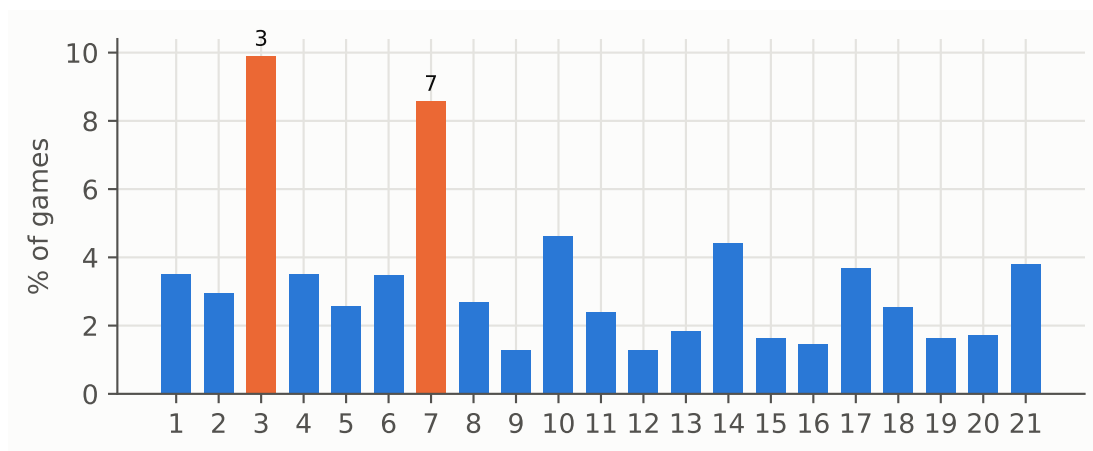


Figure 3.2: Share of games decided by exactly N points. Field goals (3) and touchdowns (7) dominate.

Margins cluster on **3 and 7**, so a normal distribution around a predicted margin isn't quite right when converting it into a probability of covering.

3.3 Scoring is not stable over time

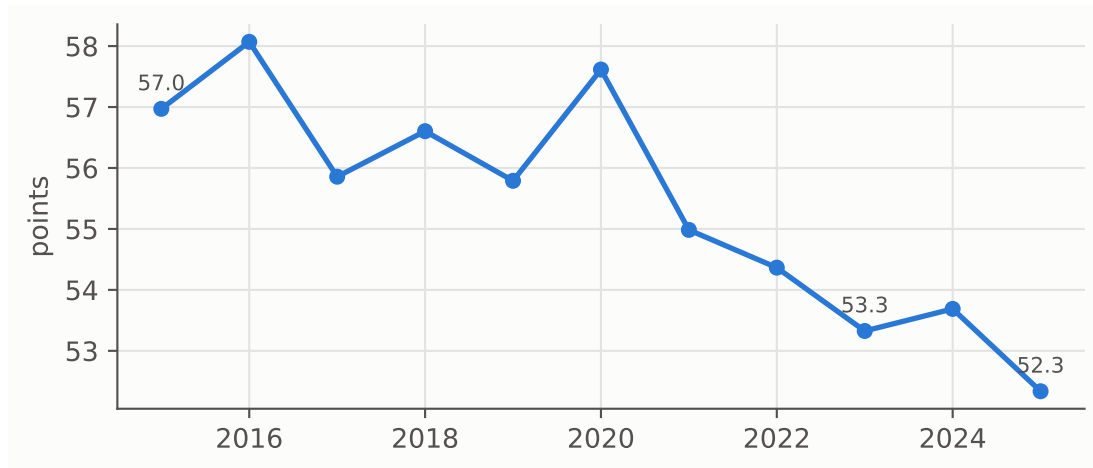


Figure 3.3: Average game total by season (FBS vs FBS). The 2023 clock rule cut plays per game.

The average total fell by about 5 points between 2015 and 2025. The 2023 rule that keeps the clock running after first downs cut plays. Two consequences for features: **normalize by the season's league average** instead of using raw points, and **weigh recent seasons more** (recency decay in the ratings, Chapter 4).

3.4 Home field and quarters

- **Home field:** home teams win by 4.3 points at their own stadium and 1.0 at neutral sites.
- **Quarters are uneven:** Q1 12.2, Q2 16.3, Q3 12.4, Q4 13.9 points per game. Q2 is the highest-scoring quarter because of end-of-half drives. Overtime happens in 4.4% of games.

Part II

Features and models

4 Feature engineering

4.1 The as-of rule

A feature for a game may only use games **played before it**. Using the game's own result, or anything later, is *leakage*: the backtest looks great and the model fails in real life.

In practice, features are computed per **slate** (season, season type, week) from earlier slates only. A Saturday game never sees Thursday's result from the same week, which also matches what's known when lines are bet midweek. The rule is enforced by a test (`tests/test_features.py`): building the features with every future result **hidden** must produce exactly the same values as building them with full data. Both the score-based and the CFBD-based features pass it.

Only FBS-vs-FBS games feed ratings and form; FCS blowouts would distort them.

4.2 Opponent-adjusted ratings

Scoring 40 against a bad defense isn't the same as scoring 40 against a good one. The core feature, following Radjewski (2021), solves for every offense and defense at once. Before each slate s , fit on prior team-games i :

$$\text{points}_i = \mu + o_{\text{team}(i)} - d_{\text{opp}(i)} + h \cdot \text{hfa}_i + \varepsilon_i, \quad \text{hfa}_i \in \{+1, 0, -1\},$$

where o is offensive strength, d defensive strength (higher is better), and h the home edge. Coefficients are estimated with **ridge regression** and **recency weights**:

$$\min_{\mu, o, d, h} \sum_i w_i (\text{points}_i - \hat{\text{points}}_i)^2 + \alpha (\|o\|^2 + \|d\|^2 + h^2), \quad w_i = 2^{-\text{age}_i / \tau},$$

with age_i in days and half-life τ . The training window is 600 days, so in September last season acts as the prior, and current form takes over as games accumulate. Ridge shrinks teams with little data toward average. The model's own prediction for a matchup is

$$\text{exp_points} = \mu + o_{\text{team}} - d_{\text{opp}} + h \cdot \text{hfa}.$$

4.2.1 Tuning α and τ

Table 4.1: MAE of exp_points alone (team points, 2017–2023) for each shrinkage α and recency half-life τ (days).

half-life alpha	60	120	240
0.3	9.96	9.59	9.61
1.0	9.84	9.55	9.56
3.0	9.94	9.69	9.61
8.0	10.27	10.04	9.87

$\alpha = 1$, $\tau = 120$ days wins. Heavy shrinkage ($\alpha = 8$) squeezes every team toward average; a short memory ($\tau = 60$) forgets last season too fast for September. An early version used $\alpha = 25$ and compressed every rating to about ± 1 point; that bug and its fix are in Chapter 9.

The same machinery is reused for **any per-game offensive stat** (plays, EPA per play, success rate, explosiveness, pass and rush EPA). The target is standardized before fitting, so α means the same shrinkage for every stat; coefficients are returned in the stat’s own units.

4.3 Elo with a nonlinear margin of victory

A second rating, FiveThirtyEight-style (FiveThirtyEight, n.d.). After each game,

$$R' = R + K \cdot M \cdot (S - E), \quad E = \frac{1}{1 + 10^{-\Delta/400}}, \quad M = \ln(|\text{MOV}| + 1) \cdot \frac{2.2}{0.001 \Delta_w + 2.2},$$

with $K = 25$, home edge 55 Elo points, and one-third regression to 1500 each offseason. The logarithm gives diminishing credit for blowouts (a 40-point win is worth little more than a 30-point win). The second factor corrects autocorrelation: favorites win by more, which would otherwise inflate their rating (Morse, n.d.).

4.4 Feature families

Table 4.2: Feature families and their sizes.

	features	what
raw_form	11	points for/against, season-to-date and last 3,...

Table 4.2: Feature families and their sizes.

	features	what
ratings	7	ridge offense/defense for team and opponent, e...
elo	3	Elo, opponent Elo, difference
matchup	9	explicit interactions: off $\times$ opp_def, expected...
context	9	neutral site, conference game, rest days and d...
efficiency	30	ridge-adjusted tempo, EPA/play, success rate, ...
pace_x_efficiency	3	expected plays $\times$ expected EPA/play, game pace,...
priors	8	talent composite and returning production, tea...

4.5 Which families earn their place?

Walk-forward cross-validation (train on every season before, validate on each of 2019–2023; Chapter 6). Families are added one at a time. Ridge is linear; “LightGBM on residual” learns `points - exp_points`, so the ratings carry the signal and the trees correct what they miss.

Table 4.3: Cross-validated MAE (points per team, 2019–2023) as feature families are added.

	Ridge	LightGBM on residual
baseline: league average	11.09	—
ratings formula alone	9.44	—
market: closing line	8.91	—
raw_form	10.02	9.44
+ratings	9.44	9.45
+elo	9.44	9.44
+matchup	9.44	9.44
+context	9.44	9.43
+efficiency	9.41	9.38
+pace_x_efficiency	9.41	9.38
+priors	9.33	9.31

Note: the “LightGBM on residual” column always starts from the ratings’ prediction (it learns `points - exp_points`), so even its `raw_form` row already contains the ratings. That’s why it starts near 9.44, not near 10.

The reading:

- **Opponent adjustment is the big win.** The league average misses by ~11.1; the ratings alone by ~9.44. Raw form (unadjusted points) only reaches ~10.0.

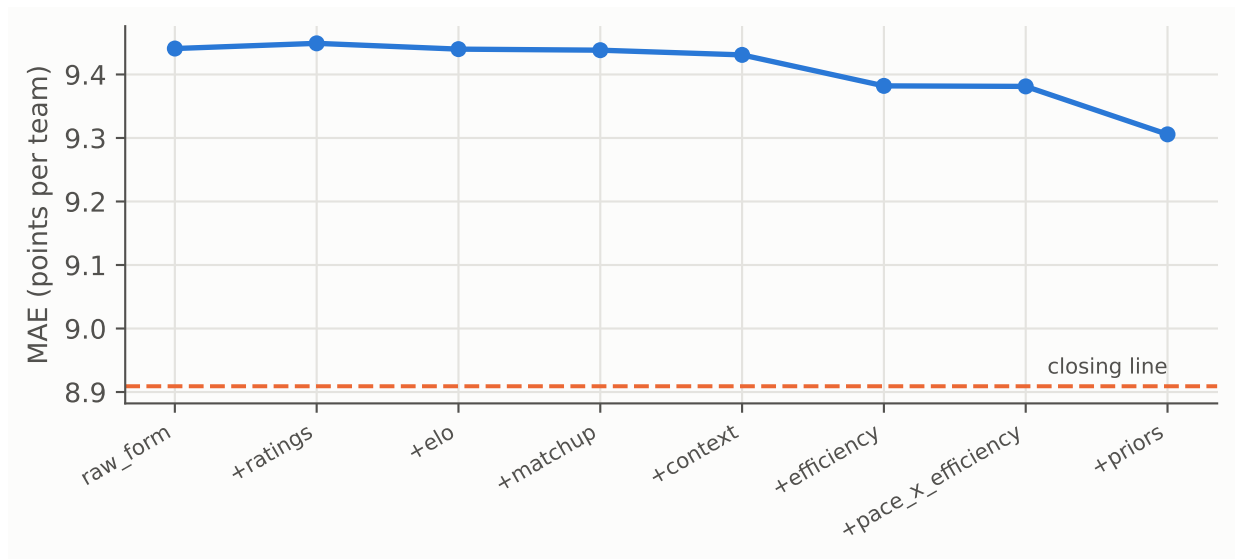


Figure 4.1: Error as families are added (LightGBM on residual), against the league-average baseline and the closing line.

- Elo, explicit matchup terms and context add almost nothing on top of the ratings.
- **CFBD efficiency** helps a bit (~9.38), and **talent and returning production** help most among the new families (~9.30), mainly by fixing early weeks.
- The explicit pace $\times$ efficiency product adds nothing: trees already combine plays and efficiency on their own.
- The **closing line** is still better (~8.91). The gap is information, not model complexity, which is what Chapter 5 investigates.

5 Nonlinearities and interactions

The feature work so far asked *which* information helps. This chapter asks *how* it helps: which relationships are curved, which have thresholds, and which variables interact. Those answers are what feature engineering should target. There are two lenses:

1. **What did the model learn?** SHAP values (Lundberg and Lee 2017) on the points model.
2. **Where is the market wrong?** Segments where the closing line misses systematically.

5.1 First, a direct test: do interactions help at all?

An additive model can't represent interactions: gradient boosting with single-split trees (stumps) builds $f(x) = \sum_j f_j(x_j)$. If deeper trees, which *can* learn interactions, don't beat stumps, the data holds no interactions a model can exploit. In every experiment (Table 5.1, copied from the experiment logs), the additive model matched or beat the deeper ones:

Table 5.1: Walk-forward CV MAE: additive vs interaction-capable LightGBM.

Feature set	Additive (stumps)	Depth 2	15 leaves
v1 (scores only)	9.460	9.482	9.607
v2 (+ CFBD efficiency, priors)	9.295	9.302	9.419

5.2 Model lens: shapes of the main effects

The model below is deliberately deeper than the tuned one (15 leaves), so interactions *can* show up if they exist.

Feature	Shape	Reading	Feature idea
Expected game total	+2 $\rightarrow$ -2 points, steepest at the ends	Ratings overstate high-scoring games: regression to the mean	Shrink extremes; a shootout flag
Talent difference	Negative tail, flat middle, positive tail	Only big talent gaps matter	Tail/threshold features matter

Feature	Shape	Reading	Feature idea
Returning production	Strongly negative in the lowest decile, flat above the median	Heavy roster turnover hurts	Low-returning-production threshold
Expected success rate	Convex, largest at the top	Elite efficiency carries information that points miss	Elite-efficiency tail

5.3 Model lens: interaction strength

Table 5.2: Strongest pairwise SHAP interactions (mean absolute contribution, points).

	feature B	strength
feature A		
exp_total	talent_diff	0.14
exp_points	talent_diff	0.14
exp_total	ret_usage	0.12
opp_std_pa_rel	talent_diff	0.11
opp_def	opp_rush_ppa_def	0.09
exp_points	exp_sr	0.09
resid_std	talent_diff	0.08
opp_std_pf_rel	talent_diff	0.07

The strongest interaction is worth about 0.14 points, against about 10 points of main effects summed. **The signal lives in one-variable curves and thresholds, not in crosses.** That agrees with Table 5.1.

5.4 Market lens: where does the closing line miss?

The market is more accurate than the model on average, but it might still be wrong in particular *places*. Every candidate feature was cut into quintiles (on 2016–2023), and each segment’s over rate (vs the closing total) or home-cover rate (vs the closing spread) was tested. Then the same segments were checked on **2024–2025**, which was never used to find them.

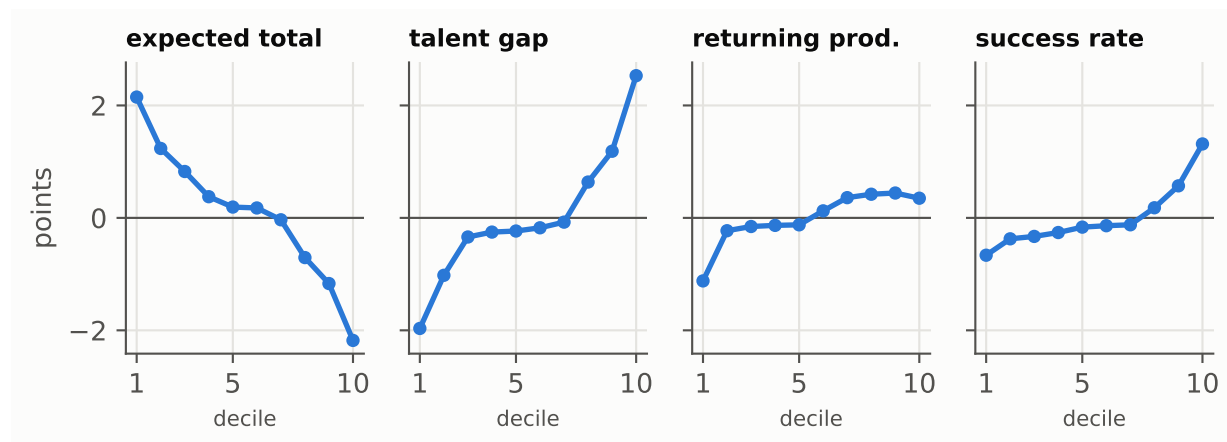


Figure 5.1: Mean SHAP value by feature decile: points the model adds to the ratings' prediction.

Table 5.3: Segments where the closing line missed in 2016–2023 ($|z| \geq 2$), with the 2024–25 holdout.

	outcome	segment	2016–23	n	z	2024–25	n hold	holds
feature								
exp_total	over %	(63.5, inf]	45.4	1181	-3.2	44.6	157	yes
game_pace	over %	(4.8, inf]	45.7	1174	-3.0	49.4	269	yes
ret_diff	home cover %	(-inf, -0.3]	46.0	1151	-2.7	53.8	292	no
total_close	over %	(62.5, inf]	46.1	1163	-2.7	45.3	86	yes
elo_diff	home cover %	(54.2, 153.0]	46.8	1176	-2.2	53.6	332	no
week	over %	(-inf, 3.0]	47.1	1343	-2.2	47.7	371	yes
week	over %	(6.0, 9.0]	47.1	1230	-2.1	52.0	329	no
ret_diff	home cover %	(0.3, inf]	53.0	1157	2.0	51.7	327	yes

39 segments were tested, so about 2 would clear $|z| \geq 2$ by chance alone. Two patterns:

- **Totals:** every surviving segment points the same way, and it holds in the holdout. High expected totals, high closing totals and very fast pace all lean **under**.
- **Spreads:** the “significant” segments flip sign in the holdout. That’s noise from testing many bins.

5.5 The shootout-under effect

Define a **shootout** as a game where the ratings' expected total is in the top 20% (above 63.5 points, a threshold set on 2016–2023).

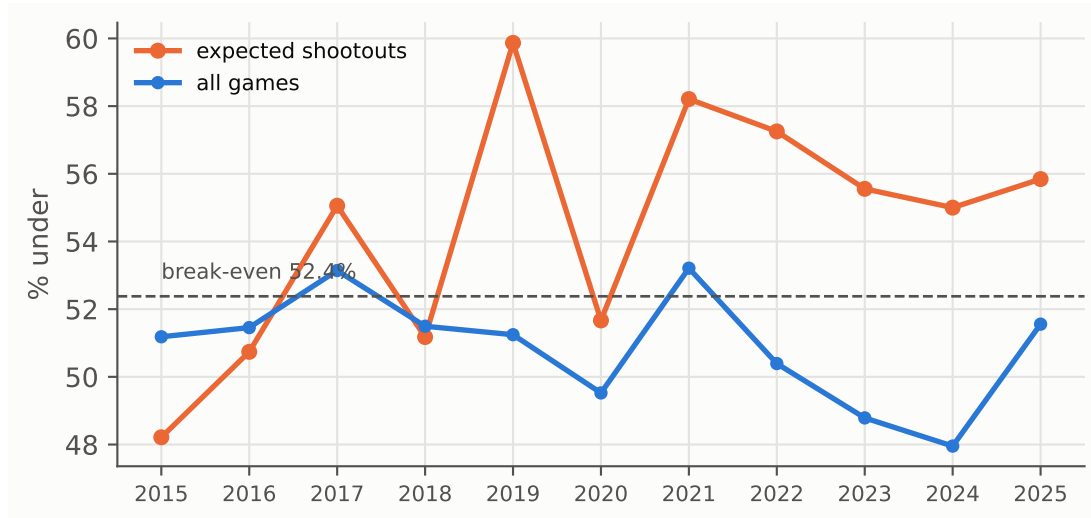


Figure 5.2: Under rate vs the closing total: expected shootouts vs all games, by season. Dashed line: 52.4% break-even at -110 .

- **Every season from 2021 to 2025** went 55–60% under in expected shootouts (2019: 59.9%; 2020, the COVID season: 51.7%; 2015–2018 near 50%).
- The gap is there **even in over-heavy years**: in 2024 only 48% of all games went under, but 55% of expected shootouts did.
- **Against the bettable opening total**, 2021–2025 pooled: 56.4% under on 509 games, above break-even in each of the five seasons.

It's the same nonlinearity as the expected-total SHAP curve, seen from the market side: **games that look like shootouts get overestimated, by the ratings and by the market**. It became paper rule B (Chapter 8). An honest caveat: the 20% cut was chosen while looking at 2016–2023, which includes 2021–2023; the clean confirmation is 2024–2025.

6 Models, tuning and the one-shot test

6.1 Splitting by time

A random 80/20 split leaks the future: the model would train on November and “predict” September, already knowing how the season turned out. So every split is **by time**:

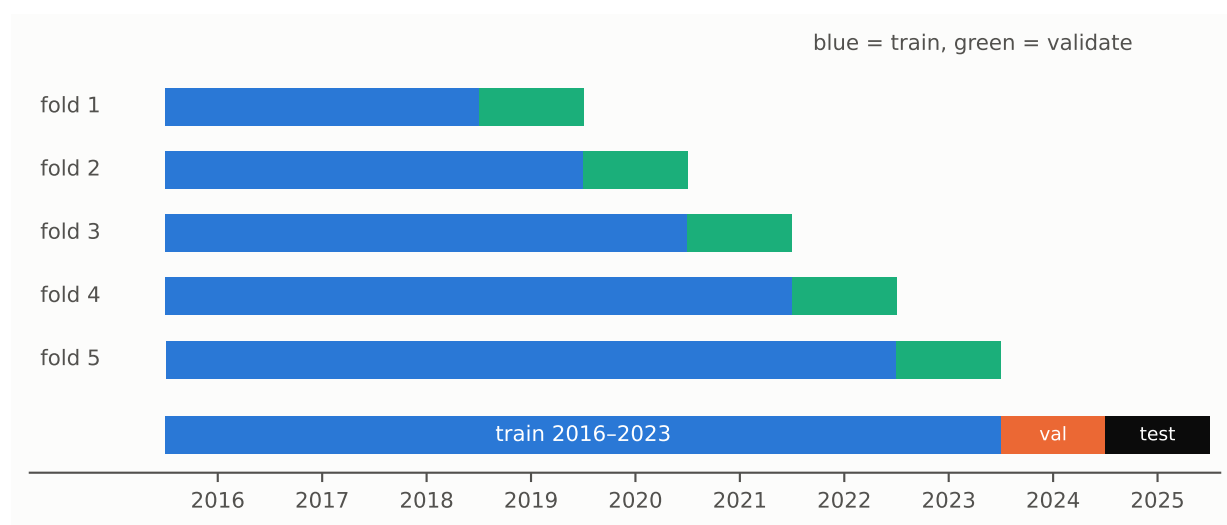


Figure 6.1: Train, validation and test by season, and the walk-forward cross-validation folds inside the training window.

Table 6.1: Data splits.

Set	Seasons	Used for
Train	2016–2023 (2015 only feeds features)	Fitting; cross-validation and Optuna run only here
Validation	2024	Choosing the final model and ensemble
Test	2025	Touched once , at the end: the honest number
Production	2026	Weekly predictions

Cross-validation is **walk-forward**: train on every season before, validate on the next, for 2019 through 2023. Each fold weighs the same.

6.2 The model zoo and tuning

Every model predicts points per team. Tree models learn the **residual of the ratings** (points — exp_points), the shape that won the ablations; linear models predict points directly. Each was tuned with Optuna’s TPE sampler (Akiba et al. 2019), minimizing walk-forward MAE, starting from hand-picked defaults.

i How tuning and cross-validation fit together

For **each of the six models**, Optuna proposes a set of hyperparameters (one *trial*). That trial is scored with **walk-forward cross-validation**: the model is trained on the seasons before 2019 and validated on 2019, retrained through 2019 and validated on 2020, and so on through 2023. The trial’s score is the average of those five errors. Optuna uses the scores to propose better trials (15 to 60 per model). The best trial per model is saved to `models/team_points_params.json`.

Only after that are the six tuned models combined and compared on 2024, and only the final choice is tested on 2025. The half and quarter models reuse LightGBM’s tuned parameters rather than being tuned separately.

Table 6.2: Optuna results (mean per-season CV MAE, 2019–2023). Read live from `models/team_points_params.json`.

	defaults	tuned	trials	gain
xgboost	9.301	9.255	40	0.045
catboost	9.274	9.257	25	0.017
lightgbm	9.280	9.260	60	0.020
ridge	9.333	9.273	25	0.060
random_forest	9.314	9.294	15	0.019
poisson	9.420	9.306	20	0.114

Every model lands within ~0.05 points of the others, and their errors are almost perfectly correlated (≥ 0.98). They see the same information and make the same mistakes, so an ensemble can only help a little. Ridge’s best α (978) sits at the edge of its search range (up to 1000), which is noted but doesn’t change the conclusion.

6.3 Ensembles, validation and the final pick

Out-of-fold predictions from the five CV folds fed three ensembles: a simple average, the average of the top 3, and **stacking** with non-negative least squares. The stacking weights (ridge 0.45, XGBoost 0.33, Poisson 0.10, LightGBM 0.06, random forest 0.04, CatBoost 0) were fit on the same out-of-fold rows, so their CV number is slightly optimistic. That’s why the final choice is made on 2024.

Table 6.3: Model selection and the one-shot test (copied from the experiment log).

Model	OOF 2019–23 (pooled)	Validation 2024	Test 2025
Random forest (final: best on 2024)	9.275	9.081	8.940
Stacking (NNLS)	9.200	9.097	
Average of all six	9.212	9.085	
XGBoost (best in CV)	9.236	9.133	
Market: opening line		8.869	8.696
Market: closing line		8.861	8.594

The final recipe, `random_forest`, won 2024 by a margin that’s within noise. Any of the top candidates would have been defensible. The **2025 test was run once**, with everything frozen beforehand.

Verdict on accuracy: the market is better by ~ 0.25 points per team, both at the open and at the close.

6.4 The market layer, and why it was dropped

Opening lines only exist from 2021, too few seasons to train a full model against the market. So the points model stays market-blind, and a tiny logistic regression (inputs: the edge vs the opener, the edge $\times$ early season, and the size of the line) was fit on 2021–2023 to learn *when* to trust the model’s disagreement with the opener.

- **Spreads:** 47–49% in the 2025 test. It doesn’t work.
- **Totals:** 55.7% in 2025 at $p \geq 0.5$. But its coefficient on the model’s edge came out **negative**, so it wasn’t using the model at all; it mostly bet unders on high totals. That was first read as a pure base-rate artifact of an under-heavy 2025. The nonlinearity analysis later showed it’s partly a stable effect (Chapter 5), which is now a separate, explicit rule.

The layer was dropped from the weekly card. Its useful part lives on as rule B.

7 Halves and quarters

Targets: each team’s points in 1H, 2H (with OT), and Q1–Q4 (regulation). A natural decomposition:

$$\text{points}_p \approx \text{exp_points} \times \text{share}_p,$$

where share_p is the fraction of a team’s points scored in period p . Two versions of the share were tried: the **league’s** share, and each team’s own **as-of** share (exponentially weighted over prior games, blended with the share the opponent’s defense usually allows). The final model uses the league share as its base, with LightGBM correcting the residual.

7.1 Accuracy by period

Table 7.1: Walk-forward CV MAE (points per team, 2019–2023) by period.

	mean points	constant	exp × league share	exp × team share	model	model vs constant %
period						
1H	14.36	7.42	6.66	6.68	6.56	-11.60
2H	13.51	6.78	6.35	6.42	6.29	-7.23
Q1	6.18	4.51	4.34	4.39	4.28	-5.23
Q2	8.18	5.25	4.90	4.94	4.83	-7.99
Q3	6.28	4.55	4.39	4.44	4.30	-5.46
Q4	6.94	4.46	4.58	4.64	4.45	-0.21

- The model beats the constant in every period, by about 12% in 1H and very little in Q4.
- **Team-specific scoring shares are worse than the league share.** How a team splits its points across quarters is mostly noise, and it regresses to the league pattern.
- **Q4 is special:** expected points × share does *worse* than a constant.

7.2 Game script: why Q4 doesn’t follow team strength

Big favorites score only about a fifth of their points in Q4: they run the clock once the game is decided. Big underdogs score close to 30% there (garbage time). Q4 points barely grow with team strength, unlike Q1–Q3.

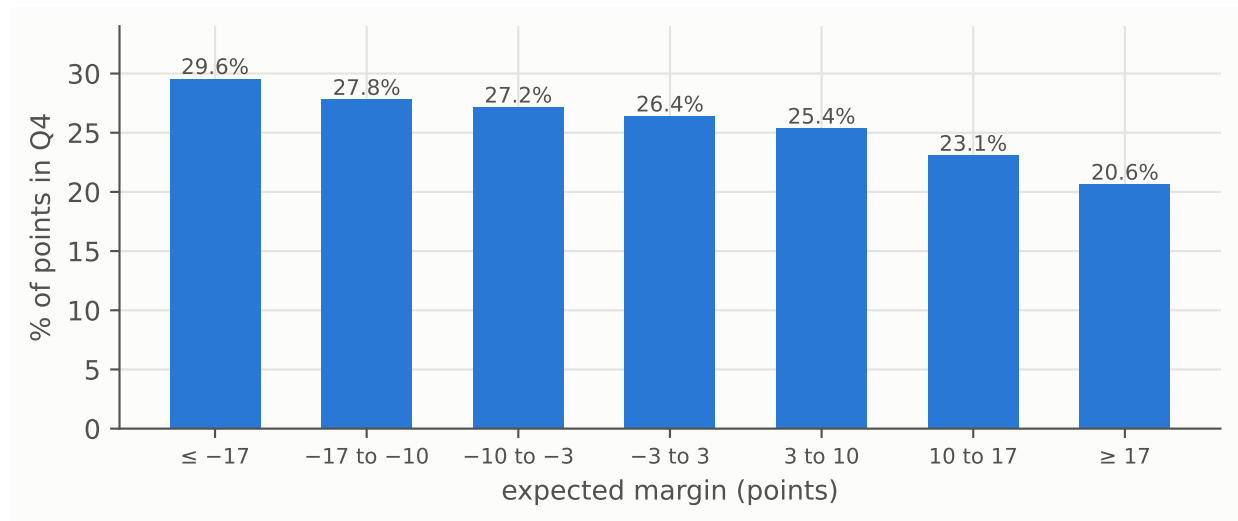


Figure 7.1: Share of a team's regulation points scored in Q4, by the team's expected margin (2019–2023).

It's a nonlinear effect of one variable (expected margin), and trees capture it directly.

7.3 Medians, not means

The period models train on absolute error, so each prediction is the period's **median**. That's the right number for an over/under (the 50/50 point), but medians don't add up: quarter predictions sum below the full-game prediction, because quarters have many zeros and a right skew. The periods aren't meant to reconcile exactly.

Status: accuracy only. There are no period betting lines yet; see Chapter 10 for the options and costs.

Part III

Betting

8 From predictions to bets

8.1 Market-implied points

A spread s (home's line) and a total T imply points for each team:

$$\text{home} = \frac{T}{2} - \frac{s}{2}, \quad \text{away} = \frac{T}{2} + \frac{s}{2}.$$

Those implied points are the benchmark the model's per-team predictions are compared to (Table 6.3).

8.2 Edges and results

From the two team predictions, $\widehat{m} = \widehat{\text{home}} - \widehat{\text{away}}$ and $\widehat{T} = \widehat{\text{home}} + \widehat{\text{away}}$. Against a line:

$$\text{spread edge} = \widehat{m} + s, \quad \text{total edge} = \widehat{T} - T.$$

A positive spread edge means the model likes home; a positive total edge means the over.

8.3 Break-even and ROI at -110

Risking 110 to win 100 pays $100/110$ per unit risked. Break-even solves $p \cdot \frac{100}{110} = (1 - p)$:

$$p^* = \frac{110}{210} \approx 52.38\%.$$

ROI per unit risked is $p \left(1 + \frac{100}{110}\right) - 1$. Pushes return the stake and are excluded from win rates.

8.4 Closing line value (CLV)

CLV measures how far the line moved *toward* a pick between when it was taken (the opener) and kickoff (the close):

$$\text{CLV}_{\text{total}} = \text{sign}(\text{edge}) \cdot (T_{\text{close}} - T_{\text{open}}).$$

Taking over 50 and seeing it close at 52 is +2: the market later agreed. Positive CLV is the most stable early sign of real edge, because it doesn't depend on the noisy outcome of the game. Across the held-out seasons, the model's picks against the opener had **positive CLV every year, growing with the size of the edge** (about +0.3 to +1.0 points in 2025).

8.5 The pre-registered paper rules

Both rules were fixed on 2026-09-23, before any 2026 prediction was graded. They're tracked on paper only.

Table 8.1: Paper-trading rules.

Rule	Bet	Evidence when registered
A • edge4	The model's side when $ \text{model total} - \text{opening total} \geq 4$	57.8% (2024, $n = 213$) and 55.2% (2025, $n = 201$); the only rule above break-even in both held-out years, within about 1 SE; threshold chosen after looking
B • shootout_under	The under when the ratings' expected total exceeds 63.5	55–60% under in every season 2021–2025; 56.4% vs the opener pooled over 2021–2025 ($n = 509$)

The rules can disagree. On day one, Ole Miss at Florida was an **over** for A (model 66.8 vs 59.2) and an **under** for B (expected total 68.5). Each rule is graded on its own, and the conflict itself points at the next piece of work (Chapter 10).

Decision rule for real money, set in advance: at least one full season of paper results, a win rate above 52.4%, and positive CLV.

8.6 Logged picks so far

Table 8.2: Paper bets logged for 2026 (before kickoff).

	week	away	home	open total	model total	edge	pick
rule							
edge4	4	Air Force	Nevada	48.8	56.8	8.0	over
edge4	4	Ole Miss	Florida	59.2	66.8	7.5	over
edge4	4	Army	Temple	49.2	56.3	7.1	over
edge4	4	South Carolina	Alabama	50.2	55.6	5.4	over
edge4	4	TCU	UCF	50.0	44.7	-5.3	under
edge4	4	UConn	Miami (OH)	50.8	55.5	4.7	over
edge4	4	Oklahoma State	West Virginia	58.5	53.9	-4.6	under
edge4	4	Central Michigan	Miami	53.8	49.3	-4.5	under
edge4	4	Massachusetts	Sacramento State	49.5	53.9	4.4	over
edge4	4	Clemson	California	49.0	53.2	4.2	over
edge4	4	Nebraska	Michigan State	50.5	54.6	4.1	over
edge4	4	Georgia Tech	Stanford	48.2	52.3	4.1	over
shootout_under	4	Notre Dame	Purdue	57.0	59.6	2.6	under
shootout_under	4	Ole Miss	Florida	59.2	66.8	7.5	under
shootout_under	4	Oregon	USC	60.2	62.8	2.6	under

Graded so far: 0 (week 4 hasn't been played as of this writing). The running record lives in `paper_trading/2026_summary.md` and is updated weekly.

8.7 The weekly pipeline

`scripts/predict_week.py` runs the whole week in about a minute:

1. Refresh the current season: ESPN scores plus CFBD lines and advanced stats (about 3 CFBD calls).
2. Rebuild the feature table.
3. Refit the final model on every completed game since 2016.
4. Predict both teams' points (and every half and quarter) for the next slate.
5. Compute edges against the opening lines, and append the rule A and B picks to `paper_trading/2026_bets.csv`.
A (game, rule) pair already logged keeps its first pick.

`scripts/grade_paper.py` grades finished games (win/loss/push at the opening total, units at -110, CLV) by rule and by week.

A **GitHub Action** (`.github/workflows/weekly.yml`) runs both every Wednesday at 13:00 UTC (9:00 ET, before the Thursday games) and commits the picks and results. That makes git's timestamps the proof that every pick came before its game. A clean-machine simulation reproduced exactly the same picks as the local run.

9 Decision log and corrections

Everything below was decided on 2026-09-23 unless noted. Corrections are listed explicitly. A project like this is only trustworthy if its mistakes are visible.

9.1 Decisions

Table 9.1: Main decisions.

Decision	Chosen	Alternatives	Why
Repository	New, separate canes-cfb-analytics	Extend cfb-canec-analytics	Notebook-first project with its own scope
Organization	One folder per market type , one notebook per period	Folder per period	Keeps models of the same kind together
Benchmark	Sportsbook lines only	Exchanges (Kalshi, Polymarket)	Scope requested; exchange references removed from the repo
First target	Points per team	Game total, spread	One model yields spread, total and winner
Split	By time: train 2016–23, validate 2024, test 2025	Random 80/20	Random splits leak the future
Data first	ESPN (free) for games and quarters, CFBD for lines and efficiency	CFBD only	Could start before the API key existed
Final points model	Random forest on the ratings residual	Stacking, XGBoost	Best on 2024 validation (within noise)
Market layer	Dropped	Keep	Spreads failed; the totals result wasn't using the model
Real money	No ; paper-trade two pre-registered rules through 2026	Bet now	Edge not proven; the market is more accurate on average

Decision	Chosen	Alternatives	Why
Weekly runs	GitHub Actions	Local cron, manual	Runs with the Mac off; commits give tamper-evident timestamps
Period lines (#4)	Build models now, buy lines later	Buy history now (\$119) or forward only (\$30/month)	See whether period models predict well first

9.2 Corrections: things that were wrong and got fixed

1. **Ratings were over-shrunk.** The first ridge ratings used $\alpha = 25$ and came out with a spread of about ± 1.3 points, when real team differences are about ± 15 . Tuning found $\alpha = 1$ (Table 4.1).
2. **A stale feature file.** The first ablation run used features built *before* the α fix, so “ratings alone” showed 10.4 instead of 9.44. The inconsistency with the tuning grid gave it away; the table was rebuilt and rerun.
3. **Notebook ordering bug.** The feature notebook once saved a v1-only table over the full one, which broke the backtest downstream. Sections were separated, and v1 experiments now use `cumulative_sets(include_v2=False)`.
4. **Lint “fixes” deleted notebook imports.** Ruff’s autofix removed “unused” imports from empty notebook skeletons. The unused-import rule is now disabled for notebooks only.
5. **“The totals result is a 2025 artifact.”** The market layer’s 55.7% on 2025 totals was first dismissed as an under-heavy year. The season-by-season analysis showed a stable shootout-under effect from 2021 to 2025. The dismissal was partly wrong, and the effect became rule B.
6. **“Since 2019.”** The shootout effect was first described as holding every season since 2019; 2020 was 51.7%. The accurate statement: every season from 2021 to 2025.
7. **Ranges in the backtest notes** were rounded too generously (for example “49–50%” when a threshold reached 48.4%). They were replaced with the exact ranges.
8. **Opening lines availability.** The early “vs open” backtest was described as covering 2019–2023; opening lines only exist from 2021, so it covered 2021–2023.

9.3 What the evidence says, plainly

- The market is **more accurate** than the model at both the open and the close.
- Spread signals **flip sign** between seasons: noise.
- Two totals signals survived a held-out check. Neither is statistically decisive yet, and both are tracked on paper.

- The model's disagreement with the opener predicts **line movement** (positive CLV) every season. That's the most robust signal found so far.

10 What's next

10.1 Open work

Table 10.1: Open work.

Issue	What	Status
#10	Totals model built on the nonlinear insights: shootout flag, pace tail, talent-gap tails, low returning production, early season. The opening total is the base	Next
#4	Betting lines for halves, quarters and team totals	Decided: models first, lines later
#8	Weather (wind thresholds of about 10 and 15 mph; Football Study Hall (2018)) via the free Open-Meteo API	Backlog
—	Winner probabilities from the predicted margin	Backlog
—	Real-money decision after a full season of paper results	2026 season end

10.2 Period lines (#4): options and costs

The only source found with historical period and team-total lines is The Odds API (*The Odds API* 2026). Its market keys include `spreads_h1`, `totals_q1 ... q4`, and `team_totals_h1`. Historical data for these “additional markets” starts on **May 3, 2023** and requires a paid plan.

Table 10.2: Options for period lines.

Option	Cost	Gives
History, one month of the 5M-credit plan	\$119 once	2023–2025 seasons: about 2,400 games $\times$ 15 markets $\times$ 10 credits $\approx$ 360K credits for one snapshot per game
Forward only, 20K-credit plan	\$30/month	Weekly capture from now on (about 900 credits/week)
Free plan (500 credits/month)	\$0	Not enough for weekly capture

One unknown: how many college games actually get quarter lines. Books tend to post them only for bigger games. That would be checked on the first pull.

10.3 Should this become a website?

This book renders as both a PDF and a website from the same source. Publishing the HTML version on GitHub Pages costs nothing extra. The next useful step would be a small page that shows the weekly card and the running paper record, updated by the same weekly Action.

A Reproducing everything

A.1 Environment

```
git clone https://github.com/DanielRegaladoUMiami/canes-cfb-analytics
cd canes-cfb-analytics
mkdir .venv.nosync && ln -s .venv.nosync .venv # only if the folder is iCloud-synced
uv sync
uv run pre-commit install
uv run pytest # all tests, offline
```

Put the CollegeFootballData key in your shell profile, never in a file inside the repo:

```
export CFBD_API_KEY=... # in ~/.zshrc
```

A.2 Order of operations

Run these in order. Notebooks live in notebooks/; scripts in scripts/.

1. **Games and quarter scores:** 00_data/01_ingest_games.ipynb → data/raw/games.parquet
2. **Lines, efficiency, priors:** 00_data/02_ingest_lines.ipynb → data/raw/*.parquet
3. **EDA:** 00_data/03_eda.ipynb
4. **Features:** 00_data/04_feature_engineering.ipynb → data/processed/team_games.parquet
5. **Nonlinearity insights:** 00_data/05_nonlinearity_insights.ipynb
6. **Tuning** (30–60 min): uv run python scripts/tune_team_points.py → models/team_points_params.json
7. **Ensemble, validation, one-shot test:** 04_team_total/full_game.ipynb → models/team_points_final.json
8. **Halves and quarters:** 04_team_total/periods.ipynb
9. **Weekly card:** uv run python scripts/predict_week.py → data/predictions/, paper_trading/
10. **Grading:** uv run python scripts/grade_paper.py → paper_trading/2026_results.csv, 2026_summary.md

The 2025 test in step 7 is meant to be run once. Re-running it reproduces the same numbers (the seeds are fixed), but changing the model after looking at it would make it no longer a test.

A.3 Rendering this book

```
cd report
export QUARTO_PYTHON=$(cd .. && pwd)/.venv/bin/python
quarto render          # HTML (report/_book/) and PDF
```

Computations are frozen (`_freeze/`), so the book re-renders without the data. To recompute a chapter after the data changes, delete its folder under `_freeze/` and render with the data present.

B Glossary

Table B.1: Terms used in this book.

Term	Meaning
As-of	Computed only from information available before the game
Ablation	Adding (or removing) one feature family at a time to measure its contribution
Break-even	The win rate needed not to lose money: 52.38% at -110
CLV (closing line value)	How far the line moved toward a pick between when it was taken and kickoff
Closing / opening line	The market's number at kickoff / when first posted
Cover	Winning against the spread: home covers when $\text{margin} + \text{spread} > 0$
Edge	Model prediction minus the market's line, in points
EPA / PPA	Expected points added per play: efficiency above what a situation usually yields
exp_points	The ridge ratings' prediction for a team's points in a matchup
FBS / FCS	College football's top division / the division below
Game script	How the score shapes play: leaders run the clock, trailers pass and score late
Leakage	Using information from the future (or the outcome itself) in a feature
MAE	Mean absolute error, in points
OOF	Out-of-fold: predictions for rows the model didn't train on
Push	Exactly on the line; the stake is returned
Ridge regression	Linear regression with an L2 penalty that shrinks coefficients toward zero
SHAP	Attribution of a prediction to each feature (and pairs, for interactions)

Term	Meaning
Shootout	A game whose expected total is in the top 20% (above 63.5 points)
Slate	One (season, season type, week)
Stacking	Combining models with weights learned from their out-of-fold predictions
Walk-forward CV	Train on all seasons before N, validate on N, repeat for several N
–110	Standard price: risk 110 to win 100

References

Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. “Optuna: A Next-Generation Hyperparameter Optimization Framework.” *KDD*.

CollegeFootballData.com API. 2026. <https://collegefootballdata.com>.

FiveThirtyEight. n.d. *How Our NFL Predictions Work*. <https://fivethirtyeight.com/methodology/how-our-nfl-predictions-work>.

Football Study Hall. 2018. *College Football Betting: How Does Wind Affect the over/Under?* <https://www.footballstudyhall.com/2018/6/25/17500384/football-betting-windy-conditions-effect>.

Lundberg, Scott M., and Su-In Lee. 2017. “A Unified Approach to Interpreting Model Predictions.” *Advances in Neural Information Processing Systems*.

Morse, Steven. n.d. *Autocorrelation in Elo Ratings*. <https://stmorse.github.io/journal/Elo-2.html>.

Radjewski, Bill. 2021. *Opponent Adjusted Stats Using Ridge Regression*. <https://radsportsanalytics.com/blog/opponent-adjusted-stats-ridge-regression/>.

The Odds API: Historical Odds Data. 2026. <https://the-odds-api.com/historical-odds-data/>.